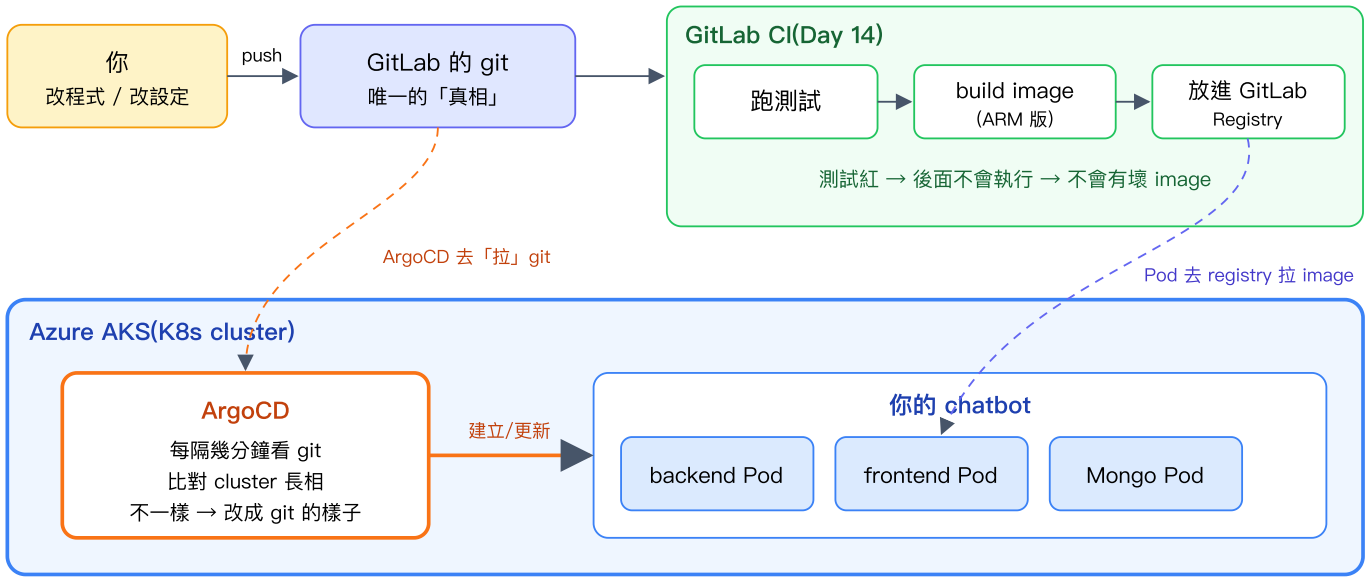
Day 15–16:GitOps 圖解

ArgoCD + GitLab CI:從 git push 到上線,不手動部署

1. 一張圖看懂今天的整條路



白話:以前你要自己下 `helm upgrade` 才會部署,做了什麼只有你記得。現在 **git 是唯一的真相**:你只管改 git,CI 負責驗證與打包,ArgoCD 負責讓 cluster 長得跟 git 一樣。

2. 你問的:ArgoCD 是管 Azure 的工具嗎?

不是。ArgoCD 不認識 Azure,也不管 Azure。它只是一個**跑在 K8s cluster 裡的程式(幾個 Pod)**,只管「cluster 裡面」的 K8s 資源(Deployment、Service、ConfigMap…)。



工具	管什麼	例子
`az`	Azure 本身(第 1 層)	`az aks stop`、建 ACR
`kubectl` / `helm`	cluster 裡的資源(第 2 層),手動	`helm upgrade`
ArgoCD	第 2 層,但**自動**、依 git 為準	git 改了就自己更新
GitLab CI	程式碼 → 測試 → image	pytest、docker build

3. 你問的:aks stop 之後 ArgoCD 也關了嗎?

是的,一起停了。ArgoCD 的 Pod 跑在 AKS 的節點上。`az aks stop` 把節點關機,上面所有 Pod(包含 ArgoCD、chatbot、Mongo)都一起停止。

aks stop 之後「停了」

- 節點(虛擬機)關機,不再收運算費
- 節點上的 Pod:ArgoCD、chatbot、Prometheus
- Prometheus 沒掛硬碟 → 歷史指標會消失
- 網站打不開

「還在」

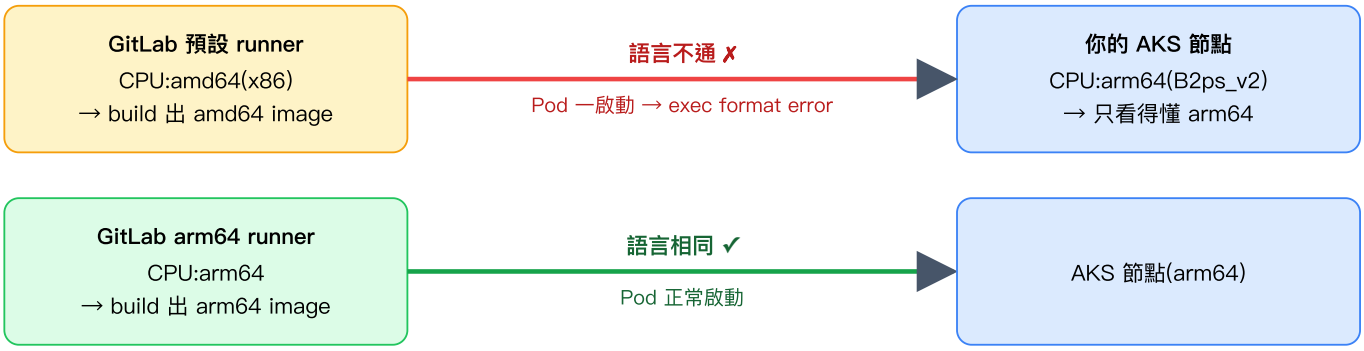
- GitLab 上的程式碼、CI、image(在雲端)
- ArgoCD 的設定(存在 cluster 資料裡)
- Mongo 的對話資料(掛了磁碟)
- 還是會計費:磁碟、ACR、公網 IP(很少)

下次 az aks start:節點開機 → Pod 自動重新啟動 → ArgoCD 起來後重新比對 git,發現一致就什麼都不用做。今天早上你就看到了這件事:啟動後 ArgoCD 直接顯示 Synced。

4. 今天發生的事(時間線)

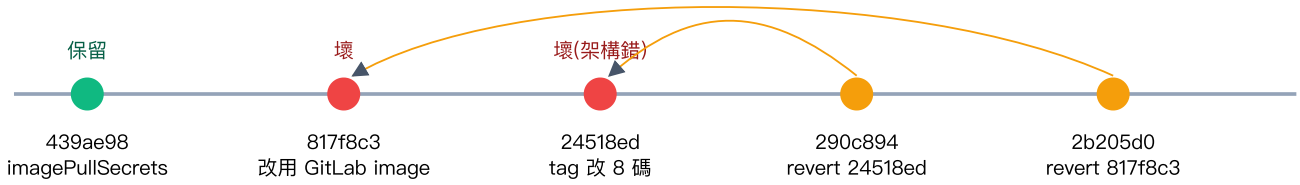
#	做了什麼	結果
1	裝 ArgoCD、建 Application 指向 GitLab 的 `mychart/`,開 auto-sync(Day 15,昨天)	Synced / Healthy
2	改 chart 讓它支援 `imagePullSecrets`(拉私有 image 用),只 git push	ArgoCD 自動滾動更新 ✅ 沒手動部署
3	把 image 改成 CI 建的,tag 填 `439ae98`	❌ `not found`
4	發現 GitLab 短 SHA 是 **8 碼**,改成 `439ae983`	image 拉到了,但 ❌ `exec format error`
5	用 `git revert` 回滾,網站恢復	Pod 全 Running ✅
6	CI 改用 **arm64 runner** 建 image	pipeline 綠 ✅
7	image 改成 `60122dd7`,git push	ArgoCD 自動部署 ✅
8	手動把前端調成 5 個 Pod(selfHeal 示範)	幾秒內被改回 2 個 ✅

5. 問題一:exec format error(CPU 架構不合)



為什麼 CI 是綠的卻部署失敗?CI 只檢查「測試過、image build 成功、推上去」,**沒有人實際執行過這個 image**。這是 CI 與實際部署之間的盲點。你以前沒遇過,是因為你在 Apple 晶片的 Mac 上自己 build,本來就是 ARM。

6. 問題二:回滾 —— 用 git revert,不是改歷史



回滾的檔案是 `mychart/values.yaml` 的 4 行(兩個 image 的路徑與 tag)。`git revert` 是「新增一個 commit 去抵銷舊的」,歷史完整保留,之後能查「當時發生了什麼」。ArgoCD 看到 git 變回舊 image,就自動把 cluster 改回去,我沒有手動動 cluster。

7. 三個 GitOps 核心功能

功能	意思	今天的證據
auto-sync	git 變了,ArgoCD 自動套用	push 後幾秒 Pod 開始滾動更新
selfHeal	有人手動改 cluster,自動改回 git 的樣子	手動調成 5 個前端 Pod → 幾秒內回到 2 個
prune	git 裡刪掉的資源,cluster 也刪	已設定(今天沒刪資源,沒實測)

> **滾動更新的保護(為什麼壞 image 沒讓網站掛掉):**K8s 規則是「新 Pod 沒 Ready,舊 Pod 就不撤」。第一次 `not found` 時舊 Pod 還在服務。第二次 `exec format error` 時有一段時間後端沒有健康 Pod,所以我立刻 revert 止血。

pull secret 是幹嘛的?

CI 建的 image 放在 GitLab 的**私有** registry。Pod 要拉就得出示憑證,`gitlab-registry` 這個 secret 就是你的唯讀 deploy token。它含機密所以只放 cluster 裡,**不進 git**。

8. 面試可以問 / 可以講的點

Q1. 什麼是 GitOps?為什麼不直接 helm upgrade?

把 git 當成唯一真相,系統自動讓環境符合 git。好處:每個變更有紀錄(誰、何時、為什麼)、可 review、回滾就是 git revert、不會有「只存在某人電腦上的設定」。

Q2. ArgoCD 是 push 還是 pull 模式?為什麼重要?

Pull:ArgoCD 在 cluster 裡面主動去 git 拿。CI 不需要持有 cluster 的管理權限,安全性較好(CI 被入侵也不能直接動 cluster)。

Q3. selfHeal 解決什麼問題?

防止 configuration drift(設定漂移)。有人半夜手動改了 cluster,ArgoCD 會改回來,所以 git 與現況不會悄悄分歧。我實際示範過:手動把前端調成 5 個,幾秒內被改回 2 個。

Q4. 你遇過什麼部署問題?怎麼處理?(STAR 好題材)

狀況:CI 全綠,但部署後 Pod 崩潰。**診斷:**`kubectl logs` 看到 `exec format error`,對照發現 CI runner 是 amd64、AKS 節點是 arm64。**止血:**git revert,ArgoCD 自動回滾,服務恢復。**根治:**CI 改用 arm64 runner 原生 build。**反思:**CI 綠不代表能跑,下一步應加 smoke test 或部署後健康檢查。

Q5. image tag 為什麼用 commit SHA,不用 latest?

SHA 可追溯:看 tag 就知道是哪個 commit,回滾精確。`latest` 是浮動的,K8s 看不出內容變了,不會重新部署,也無法回滾到「上一版」。

Q6. 為什麼不直接把 ArgoCD 的 Application 也放進 Helm?

(進階)Application 本身是「啟動 GitOps 的起點」,只 `kubectl apply` 一次(bootstrap),之後所有變更都走 git。App-of-Apps 模式可以把這件事也納入 git,目前沒做。

Q7. 回滾怎麼做?為什麼用 revert 而不是 reset?

`git revert` 新增反向 commit,歷史不被改寫,多人協作與 ArgoCD 追蹤都安全。`reset --hard` + force push 會改寫歷史,別人的副本會衝突。

9. 誠實的限制(面試被追問時直接講)

- **換版本仍需手動改 tag。**CI 建好 image 後,還沒有自動把新 tag 寫回 git;目前是我手動改 `values.yaml`。Day 22–24 整合時要自動化這一步(CI 自動 commit tag,或用 ArgoCD Image Updater)。
- **沒做部署後的自動驗證。**CI 綠不代表 image 能跑(今天就被這個坑到)。應加 smoke test。
- **pull secret 是手動建的**,不在 git 裡。正式環境會用 Sealed Secrets / External Secrets 管理。
- **ArgoCD 用內建 admin 帳號、沒接 SSO**,只經 port-forward 存取,沒有對外開放。
- **沒有實測 prune**(刪資源會自動刪除)。
- **目前 ArgoCD 偶爾顯示 Degraded**:是 HPA 在叢集剛啟動時還抓不到 CPU 指標,指標暖機後會恢復,不是新 image 的問題。

10. 費用提醒

AKS 已執行 `az aks stop`,狀態為 Stopped,不再收節點運算費。仍會計費的有:節點的磁碟、Mongo 的 5Gi PVC、ACR、公網 IP,金額小。再開用 `az aks start -g rg-chatbot-dev -n aks-chatbot-dev`。